```
"""Settings store — persistent JSON-backed user preferences.

Expanded from the original 4-field store to a full settings schema covering
Display, Audio, Gameplay, Graphics, Controls, and Accessibility. The load()
function merges saved values over a default dict, so missing keys (from old
save files) always fall back to sane defaults — no migration step needed.
"""
import json
import os

SAVE_FILE = os.path.join(os.path.abspath("."), "settings.json")

# — Default settings —
# Every key here MUST have a default; load() only overrides keys that exist
# in the saved JSON, so adding new keys later is always backward-compatible.
_DEFAULTS = {
    # — Display —
    "display_mode": 0,          # 0=windowed, 1=fullscreen
    "resolution_idx": 0,        # index into SUPPORTED_RESOLUTIONS
    # — Audio —
    "master_volume": 80,
    "music_volume": 70,
    "sfx_volume": 85,
    "mute_on_focus_loss": True,
    # — Gameplay —
    "crosshair": True,
    "crosshair_style": 0,       # 0=small, 1=medium, 2=large
    "auto_reload": True,
    "aim_assist": False,
    "subtitles": True,
    # — Graphics —
    "brightness": 50,           # 0-100, 50 = neutral
    "screenshake": True,
    "gore": True,
    "particle_density": 75,     # 0-100, scales particle counts
    # — Controls —
    "mouse_sensitivity": 50,    # 0-100, 50 = default
    "invert_mouse": False,
    # — Accessibility —
    "reduced_motion": False,
    "colorblind_mode": 0,       # 0=off, 1=protanopia, 2=deuteranopia, 3=tritanopia
}

def load():
    """Load settings from disk, merged over defaults.

    Always returns a complete dict with every key from _DEFAULTS, even if the
    save file is missing, corrupt, or from an older version with fewer keys.
    """
    result = dict(_DEFAULTS)
    if not os.path.exists(SAVE_FILE):
        return result
    try:
        with open(SAVE_FILE, "r", encoding="utf-8") as f:
            saved = json.load(f)
        if isinstance(saved, dict):
            for key, default_val in _DEFAULTS.items():
                if key in saved:
                    val = saved[key]
                    # Type-check: keep the default if the saved value is the
                    # wrong type (prevents crashes from a hand-edited file).
                    if isinstance(default_val, bool) and isinstance(val, bool):
                        result[key] = val
                    elif isinstance(default_val, int) and isinstance(val, int) and not isinstance(val, bool):
                        result[key] = val
                    # else: keep default
    except Exception:
        pass  # Corrupt file — silently use defaults
    return result

def save(settings_dict):
    """Save settings to disk as pretty-printed JSON."""
    try:
        with open(SAVE_FILE, "w", encoding="utf-8") as f:
            json.dump(settings_dict, f, indent=4, sort_keys=True)
    except Exception as e:
        print(f"Settings save failed: {e}")
```